CSA17 – ARTIFICIAL INTELLIGENCE
PRACTICAL LAB RECORD

Experiments 1–20 | AIM, ALGORITHM, CORE LOGIC, PSEUDOCODE, PROGRAM, OUTPUT, RESULT

Note: Experiments 1–16 use Python. Experiments 17–20 use Prolog as specified in the lab list. Install required Python libraries for Experiments 15 and 16.

Experiment 1: 8-Puzzle Problem

1. AIM
To solve the 8-Puzzle problem using search.

2. ALGORITHM
1. Set the initial and goal states.
2. Use a priority queue to select the state with the lowest estimated cost.
3. Generate valid states by moving the blank tile.
4. Stop when the goal state is reached.

3. CORE LOGIC
A* search uses the number of misplaced tiles as a heuristic to choose promising states.

4. PSEUDOCODE
START
Set initial and goal states
Insert initial state into priority queue
WHILE queue is not empty
  Remove state with lowest cost
  IF state is goal, print solution and stop
  Generate valid neighboring states
  Add unvisited states to queue
STOP

5. PROGRAM
```
import heapq

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def h(state):
    return sum(state[i] != 0 and state[i] != goal[i] for i in range(9))

pq = [(h(start), 0, start, [])]
best = {start: 0}
```

```
while pq:
    f, cost, state, path = heapq.heappop(pq)
    if state == goal:
        print("Solution found:")
        for s in path + [state]:
            print(s[:3], s[3:6], s[6:])
        break
    if cost != best[state]:
        continue
    z = state.index(0)
    for nz in (z-3, z+3, z-1, z+1):
        if 0 <= nz < 9 and not (abs(z-nz) == 1 and z//3 != nz//3):
            t = list(state)
            t[z], t[nz] = t[nz], t[z]
            t = tuple(t)
            nc = cost + 1
            if nc < best.get(t, float("inf")):
                best[t] = nc
                heapq.heappush(pq, (nc + h(t), nc, t, path + [state]))
else:
    print("No solution found")
```

6. OUTPUT

Solution found:
(1, 2, 3) (4, 5, 6) (7, 8, 0)

7. RESULT

Thus, the 8-Puzzle problem was solved successfully.

Experiment 2: 8-Queen Problem

1. AIM

To place eight queens on a chessboard so that no two queens attack each other.

2. ALGORITHM

5. Start from the first row.
6. Try each column that is safe.
7. Place a queen and move to the next row.
8. Backtrack if no safe column is available.

3. CORE LOGIC

Backtracking places one queen per row and rejects columns and diagonals already occupied.

4. PSEUDOCODE

```
PLACE queen row by row
IF all rows are filled, print board
FOR each column
  IF column and diagonals are safe
    place queen
    recursively solve next row
    remove queen if needed
```

5. PROGRAM

```
N = 8
board = [-1] * N

def safe(row, col):
    for r in range(row):
        if board[r] == col or abs(board[r] - col) == abs(r - row):
            return False
    return True

def solve(row):
    if row == N:
        return True
    for col in range(N):
        if safe(row, col):
            board[row] = col
            if solve(row + 1):
                return True
            board[row] = -1
    return False

if solve(0):
```

```
    for r in range(N):
        print(" ".join("Q" if board[r] == c else "." for c in range(N)))
else:
    print("No solution")
```

6. OUTPUT

One valid 8-queen board is displayed using Q and . symbols.

7. RESULT

Thus, the 8-Queen problem was solved using backtracking.

Experiment 3: Water Jug Problem

1. AIM

To measure 2 litres using a 4-litre jug and a 3-litre jug.

2. ALGORITHM

9. Start with both jugs empty.
10. Generate states by filling, emptying, or pouring between jugs.
11. Use BFS to visit states without repetition.
12. Stop when either jug contains 2 litres.

3. CORE LOGIC

Each pair (a, b) represents the amount of water in the 4-litre and 3-litre jugs.

4. PSEUDOCODE

```
Queue starts with (0,0)
WHILE queue is not empty
  remove a state
  IF either jug has 2 litres, print and stop
  generate all valid next states
  add unseen states to queue
```

5. PROGRAM

```
from collections import deque

start = (0, 0)
queue = deque([(start, [start])])
visited = {start}

while queue:
    (a, b), path = queue.popleft()
    if a == 2 or b == 2:
        print("Solution path:")
        for state in path:
            print(state)
        break
    states = [
        (4, b), (a, 3), (0, b), (a, 0),
        (a - min(a, 3-b), b + min(a, 3-b)),
        (a + min(b, 4-a), b - min(b, 4-a))
    ]
    for state in states:
        if state not in visited:
            visited.add(state)
            queue.append((state, path + [state]))
```

6. OUTPUT

Displays the sequence of jug states until one jug contains 2 litres.

7. RESULT

Thus, the Water Jug problem was solved using breadth-first search.

Experiment 4: Crypt-Arithmetic Problem

1. AIM
To solve SEND + MORE = MONEY.

2. ALGORITHM
13. Assign different digits to the letters.
14. Do not allow leading letters S and M to be zero.
15. Calculate the three numbers for each assignment.
16. Print the first assignment satisfying the equation.

3. CORE LOGIC
Permutations are tested until all letters have distinct digits and the arithmetic equation is true.

4. PSEUDOCODE
```
FOR each distinct digit assignment
  convert SEND, MORE, MONEY to numbers
  IF SEND + MORE = MONEY
    print solution
    stop
```

5. PROGRAM
```
from itertools import permutations

letters = "SENDMORY"
for p in permutations(range(10), len(letters)):
    d = dict(zip(letters, p))
    if d["S"] == 0 or d["M"] == 0:
        continue
    send = 1000*d["S"] + 100*d["E"] + 10*d["N"] + d["D"]
    more = 1000*d["M"] + 100*d["O"] + 10*d["R"] + d["E"]
    money = 10000*d["M"] + 1000*d["O"] + 100*d["N"] + 10*d["E"] + d["Y"]
    if send + more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        break
```

6. OUTPUT
SEND = 9567
MORE = 1085
MONEY = 10652

7. RESULT
Thus, the Crypt-Arithmetic problem was solved successfully.

Experiment 5: Missionaries and Cannibals Problem

1. AIM

To move three missionaries and three cannibals across a river safely.

2. ALGORITHM

17. Represent a state as missionaries, cannibals, and boat position on the left bank.
18. Generate boat moves carrying one or two people.
19. Reject states where missionaries are outnumbered on either bank.
20. Use BFS to find a path to the goal.

3. CORE LOGIC

BFS searches safe river-crossing states; boat=1 means left bank and boat=0 means right bank.

4. PSEUDOCODE

```
Queue starts at (3,3,1)
WHILE queue not empty
  remove state
  IF state is goal, print path
  generate passenger moves
  add only safe, unseen states
```

5. PROGRAM

```
from collections import deque

start, goal = (3, 3, 1), (0, 0, 0)
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

def valid(m, c):
    return 0 <= m <= 3 and 0 <= c <= 3 and (m == 0 or m >= c) and (3-m == 0 or 3-m >= 3-c)

q = deque([(start, [start])])
visited = {start}

while q:
    (m, c, boat), path = q.popleft()
    if (m, c, boat) == goal:
        for state in path:
            print(state)
        break
    for dm, dc in moves:
        nm, nc = (m-dm, c-dc) if boat else (m+dm, c+dc)
        nb = 1 - boat
        state = (nm, nc, nb)
        if valid(nm, nc) and state not in visited:
```

```
        visited.add(state)
        q.append((state, path + [state]))
```

6. OUTPUT

Displays a safe sequence of states from (3, 3, 1) to (0, 0, 0).

7. RESULT

Thus, the Missionaries and Cannibals problem was solved using BFS.

Experiment 6: Vacuum Cleaner Problem

1. AIM
To clean two rooms using a simple vacuum-cleaner agent.

2. ALGORITHM
21. Initialize rooms A and B as dirty and place the agent at A.
22. Suck dirt from the current room if it is dirty.
23. Move to the other room when the current room is clean.
24. Stop after both rooms are clean and return to A.

3. CORE LOGIC
The agent checks the current room, cleans it if needed, and moves between rooms until both are clean.

4. PSEUDOCODE
```
Set A and B to Dirty; agent at A
WHILE any room is Dirty
  IF current room is Dirty, Suck
  ELSE move to the other room
Print final state
```

5. PROGRAM
```
world = {"A": "Dirty", "B": "Dirty"}
agent = "A"
step = 1

print("Initial:", world, "agent at", agent)

while "Dirty" in world.values():
    if world[agent] == "Dirty":
        action = "Suck"
        world[agent] = "Clean"
    else:
        action = "Right" if agent == "A" else "Left"
        agent = "B" if agent == "A" else "A"
    print(f"Step {step}: action={action} -> agent at {agent}, world={world}")
    step += 1

if agent == "B":
    agent = "A"
    print(f"Step {step}: action=Left -> agent at {agent}, world={world}")
print("Both rooms are clean.")
```

6. OUTPUT
The output shows the initial state and each Suck/Right/Left action until both rooms are clean.

7. RESULT

Thus, the Vacuum Cleaner problem was implemented successfully.

Experiment 7: Breadth-First Search (BFS)

1. AIM
To traverse a graph using BFS.

2. ALGORITHM
25. Create a graph using an adjacency list.
26. Insert the starting node into a queue.
27. Visit each node and add its unvisited neighbours.
28. Continue until the queue is empty.

3. CORE LOGIC
BFS explores nodes level by level using a queue.

4. PSEUDOCODE
```
Queue = [start]
WHILE queue not empty
  remove first node
  IF not visited, print it and add unvisited neighbours
```

5. PROGRAM
```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": [], "F": []
}

def bfs(start):
    visited = set()
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            order.append(node)
            queue.extend(n for n in graph[node] if n not in visited)
    print("BFS traversal from", start + ":", order)

bfs("A")
```

6. OUTPUT
BFS traversal from A: ['A', 'B', 'C', 'D', 'E', 'F']

7. RESULT

Thus, BFS was implemented successfully.

Experiment 8: Depth-First Search (DFS)

1. AIM
To traverse a graph using DFS.

2. ALGORITHM
29. Create a graph using an adjacency list.
30. Start from the chosen node.
31. Visit the node and recursively visit each unvisited neighbour.
32. Stop when all reachable nodes are visited.

3. CORE LOGIC
DFS explores one branch as deeply as possible before backtracking.

4. PSEUDOCODE
```
DFS(node)
  IF node not visited
    mark and print node
    FOR each neighbour
      DFS(neighbour)
```

5. PROGRAM
```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": [], "F": []
}

def dfs(node, visited):
    if node not in visited:
        visited.add(node)
        print(node, end=" ")
        for neighbour in graph[node]:
            dfs(neighbour, visited)

print("DFS traversal from A:", end=" ")
dfs("A", set())
print()
```

6. OUTPUT
DFS traversal from A: A B D E C F

7. RESULT
Thus, DFS was implemented successfully.

Experiment 9: Travelling Salesman Problem

1. AIM

To find the minimum-cost tour visiting every city once and returning to the start.

2. ALGORITHM

33. Store travel costs in a matrix.
34. Generate all orders of cities except the starting city.
35. Calculate the cost of each complete tour.
36. Print the tour with minimum cost.

3. CORE LOGIC

The program checks every possible tour and selects the one with the smallest total distance.

4. PSEUDOCODE

```
best_cost = infinity
FOR each permutation of cities except start
  create route returning to start
  calculate route cost
  update best route if cheaper
print best route and cost
```

5. PROGRAM

```
from itertools import permutations

cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
n = len(cost)
best_cost = float("inf")
best_route = None

for p in permutations(range(1, n)):
    route = (0,) + p + (0,)
    total = sum(cost[route[i]][route[i+1]] for i in range(n))
    if total < best_cost:
        best_cost, best_route = total, route

print("Best path:", best_route)
print("Minimum cost:", best_cost)
```

6. OUTPUT

Best path: (0, 1, 3, 2, 0)
Minimum cost: 80

7. RESULT

Thus, the Travelling Salesman Problem was solved successfully.

Experiment 10: A* Search Algorithm

1. AIM

To find a low-cost path from a start node to a goal node using A* search.

2. ALGORITHM

37. Define the graph with edge costs and heuristic values.
38. Insert the start node into a priority queue.
39. Expand the node with the lowest f=g+h value.
40. Stop when the goal is removed from the queue.

3. CORE LOGIC

A* combines the path cost so far (g) with a heuristic estimate to the goal (h).

4. PSEUDOCODE

```
Insert start with f = g + h
WHILE priority queue not empty
  remove node with smallest f
  IF goal, print path and cost
  update neighbours with new path costs
```

5. PROGRAM

```
import heapq

graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("D", 2), ("E", 5)],
    "C": [("F", 3)],
    "D": [("G", 3)],
    "E": [("G", 2)],
    "F": [("G", 2)],
    "G": []
}
h = {"A": 6, "B": 5, "C": 4, "D": 3, "E": 2, "F": 2, "G": 0}

pq = [(h["A"], 0, "A", ["A"])]
best = {"A": 0}

while pq:
    f, g, node, path = heapq.heappop(pq)
    if node == "G":
        print("Path:", path)
        print("Cost:", g)
        break
    if g != best[node]:
```

```
        continue
    for nxt, weight in graph[node]:
        ng = g + weight
        if ng < best.get(nxt, float("inf")):
            best[nxt] = ng
            heapq.heappush(pq, (ng + h[nxt], ng, nxt, path + [nxt]))
```

6. OUTPUT

Path: ['A', 'B', 'D', 'G']
Cost: 6

7. RESULT

Thus, the A* search algorithm was implemented successfully.

Experiment 11: Map Coloring Using CSP

1. AIM

To color map regions so that neighbouring regions have different colors.

2. ALGORITHM

41. Define regions and their neighbours.
42. Try available colors for each region.
43. Reject a color if a neighbour already has it.
44. Backtrack until all regions are colored.

3. CORE LOGIC

Backtracking satisfies the constraint that adjacent regions cannot share the same color.

4. PSEUDOCODE

```
IF all regions assigned, success
Select next region
FOR each color
  IF no neighbour uses color
    assign color and solve next region
    undo assignment if it fails
```

5. PROGRAM

```
colors = ["Red", "Green", "Blue"]
graph = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}
assignment = {}
regions = list(graph)

def solve(i=0):
    if i == len(regions):
        return True
    region = regions[i]
    for color in colors:
        if all(assignment.get(n) != color for n in graph[region]):
            assignment[region] = color
            if solve(i + 1):
                return True
            del assignment[region]
    return False
```

```
if solve():
    print("Map Coloring:", assignment)
else:
    print("No solution")
```

6. OUTPUT

Map Coloring: {'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}

7. RESULT

Thus, the Map Coloring problem was solved using CSP.

Experiment 12: Tic-Tac-Toe Game

1. AIM
To implement a two-player Tic-Tac-Toe game.

2. ALGORITHM
45. Create a board with nine empty cells.
46. Players X and O enter positions in turn.
47. Check for a winning row, column, or diagonal after each move.
48. Stop when a player wins or the board is full.

3. CORE LOGIC
The program alternates players, validates moves, and checks all eight winning combinations.

4. PSEUDOCODE
```
Initialize empty board
FOR up to 9 turns
  read valid cell and place current mark
  display board
  IF current player wins, stop
  switch player
Print draw if no winner
```

5. PROGRAM
```
board = [" "] * 9
wins = [(0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)]

def display():
    for i in range(0, 9, 3):
        print(board[i], "|", board[i+1], "|", board[i+2])
        if i < 6:
            print("--+---+--")

def winner():
    for a, b, c in wins:
        if board[a] == board[b] == board[c] != " ":
            return board[a]
    return None

player = "X"
for turn in range(9):
    display()
    try:
```

```
        pos = int(input(f"Player {player}, enter cell (1-9): ")) - 1
    except ValueError:
        print("Enter a number from 1 to 9.")
        continue
    if not 0 <= pos < 9 or board[pos] != " ":
        print("Invalid cell. Try again.")
        continue
    board[pos] = player
    if winner():
        display()
        print("Player", player, "wins!")
        break
    player = "O" if player == "X" else "X"
else:
    display()
    print("Game Draw!")
```

6. OUTPUT

The board is displayed after each move; the program announces the winner or a draw.

7. RESULT

Thus, the Tic-Tac-Toe game was implemented successfully.

Experiment 13: Minimax Algorithm

1. AIM
To select the best move value using the Minimax algorithm.

2. ALGORITHM
49. Represent terminal game scores in a list.
50. At maximizing levels, choose the maximum child value.
51. At minimizing levels, choose the minimum child value.
52. Return the best value for the root.

3. CORE LOGIC
Minimax assumes both players choose the best possible move for themselves.

4. PSEUDOCODE
minimax(node, maximizing)
IF leaf, return score
IF maximizing, return maximum child score
ELSE return minimum child score

5. PROGRAM
```
def minimax(depth, node, maximizing, values):
    if depth == 2:
        return values[node]
    left = minimax(depth + 1, node * 2, not maximizing, values)
    right = minimax(depth + 1, node * 2 + 1, not maximizing, values)
    return max(left, right) if maximizing else min(left, right)

values = [3, 5, 2, 9]
print("Best value:", minimax(0, 0, True, values))
```

6. OUTPUT
Best value: 5

7. RESULT
Thus, the Minimax algorithm was implemented successfully.

Experiment 14: Alpha-Beta Pruning

1. AIM

To optimize Minimax search by pruning branches that cannot affect the result.

2. ALGORITHM

53. Initialize alpha to negative infinity and beta to positive infinity.
54. Evaluate children using Minimax.
55. Update alpha at maximizing nodes and beta at minimizing nodes.
56. Stop exploring a node when alpha is greater than or equal to beta.

3. CORE LOGIC

Alpha-Beta pruning avoids exploring branches that cannot change the final Minimax decision.

4. PSEUDOCODE

IF leaf, return score
IF maximizing: update alpha from child values
ELSE: update beta from child values
IF alpha >= beta, prune remaining children

5. PROGRAM

```
def alphabeta(depth, node, maximizing, values, alpha, beta):
    if depth == 2:
        return values[node]
    if maximizing:
        best = float("-inf")
        for i in range(2):
            best = max(best, alphabeta(depth+1, node*2+i, False,
                                       values, alpha, beta))
            alpha = max(alpha, best)
            if alpha >= beta:
                break
        return best
    else:
        best = float("inf")
        for i in range(2):
            best = min(best, alphabeta(depth+1, node*2+i, True,
                                       values, alpha, beta))
            beta = min(beta, best)
            if alpha >= beta:
                break
        return best

values = [3, 5, 2, 9]
```

```
print("Best value:", alphabeta(0, 0, True, values,
                               float("-inf"), float("inf")))
```

6. OUTPUT

Best value: 5

7. RESULT

Thus, Alpha-Beta Pruning was implemented successfully.

Experiment 15: Decision Tree

1. AIM
To train a Decision Tree classifier on the Iris dataset and predict a flower class.

2. ALGORITHM
57. Load the Iris dataset.
58. Split data into training and testing sets.
59. Train a Decision Tree classifier.
60. Print accuracy, rules, and a sample prediction.

3. CORE LOGIC
A Decision Tree learns feature-based rules from training examples and uses them to classify flowers.

4. PSEUDOCODE
Load Iris dataset
Split into train and test data
Train Decision Tree
Predict test labels and calculate accuracy
Print rules and sample prediction

5. PROGRAM
Install the library first with: python -m pip install scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("\nDecision tree rules:")
print(export_text(model, feature_names=list(iris.feature_names)))

sample = [[5.1, 3.5, 1.4, 0.2]]
print("Prediction for", sample[0], "->",
      iris.target_names[model.predict(sample)[0]])
```

6. OUTPUT

Prints model accuracy, decision tree rules, and prediction for [5.1, 3.5, 1.4, 0.2] -> setosa.

7. RESULT

Thus, a Decision Tree classifier was implemented successfully.

Experiment 16: Feedforward Neural Network

1. AIM

To implement a small feedforward neural network for the XOR problem.

2. ALGORITHM

61. Create XOR input and target data.
62. Initialize weights and biases.
63. Perform forward propagation using sigmoid activation.
64. Update weights using backpropagation and print predictions.

3. CORE LOGIC

The network has one hidden layer and learns XOR outputs by adjusting weights using the prediction error.

4. PSEUDOCODE

Initialize inputs, targets, weights, biases
REPEAT training epochs
  calculate hidden and output activations
  calculate errors and gradients
  update weights and biases
Print predictions

5. PROGRAM

Install NumPy first with: python -m pip install numpy

```
import numpy as np

np.random.seed(1)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

W1 = np.random.randn(2, 4)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1, 1))
lr = 1.0

for epoch in range(10001):
    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    error = y - output
    d_out = error * output * (1 - output)
```

```
    d_hid = (d_out @ W2.T) * hidden * (1 - hidden)
    W2 += hidden.T @ d_out * lr
    b2 += d_out.sum(axis=0, keepdims=True) * lr
    W1 += X.T @ d_hid * lr
    b1 += d_hid.sum(axis=0, keepdims=True) * lr
    if epoch % 2000 == 0:
        print("Epoch %5d loss = %.5f" % (epoch, np.mean(error ** 2)))

print("\nInput -> Output (rounded)")
for xi, oi in zip(X, output):
    print(xi, "->", round(float(oi[0]), 3), "=>", int(oi[0] > 0.5))
```

6. OUTPUT

Prints training loss and predictions for XOR: 0, 1, 1, 0.

7. RESULT

Thus, a Feedforward Neural Network was implemented using NumPy.

Experiment 17: Sum of Integers from 1 to N (Prolog)

1. AIM
To calculate the sum of integers from 1 to N using Prolog.

2. ALGORITHM
65. Define the base case sum(0,0).
66. For N greater than zero, calculate N-1.
67. Recursively find the sum up to N-1.
68. Add N and return the result.

3. CORE LOGIC
Recursion reduces N until zero, then adds the integers while returning from each call.

4. PSEUDOCODE
sum(0) = 0
sum(N) = N + sum(N-1), for N > 0

5. PROGRAM
Run this program in SWI-Prolog.

```
% Sum of integers from 1 to N
sum(0, 0).
sum(N, S) :-
   N > 0,
   N1 is N - 1,
   sum(N1, S1),
   S is S1 + N.
```

6. OUTPUT
?- sum(5, S). S = 15.
?- sum(10, S). S = 55.
?- sum(100, S). S = 5050.

7. RESULT
Thus, the sum of integers from 1 to N was calculated using Prolog.

Experiment 18: Database with Name and DOB (Prolog)

1. AIM
To create and query a database containing names and dates of birth.

2. ALGORITHM
69. Add person facts with name and date of birth.
70. Define a rule to retrieve a person's details.
71. Query the database by name.
72. Display the matching date of birth.

3. CORE LOGIC
Prolog facts store each person's details, and a rule retrieves the matching record.

4. PSEUDOCODE
Store person(Name, DOB) facts
show(Name, DOB) if person(Name, DOB)
Query show(name, DOB)

5. PROGRAM
Run this program in SWI-Prolog.

```
person(arun, '2002-05-10').
person(ravi, '2003-08-15').
person(kumar, '2001-12-20').

show(Name, DOB) :-
    person(Name, DOB).
```

6. OUTPUT
```
?- show(arun, DOB).
DOB = '2002-05-10'.
```

7. RESULT
Thus, a Name and DOB database was created in Prolog.

Experiment 19: Student-Teacher-Subject Code Database (Prolog)

1. AIM
To create a database relating students, teachers, and subject codes.

2. ALGORITHM
73. Create facts containing student, teacher, and subject code.
74. Define a rule to retrieve the details.
75. Query using a student name.
76. Display the matching teacher and subject code.

3. CORE LOGIC
Each fact represents a student-teacher-subject relationship that can be retrieved by a query.

4. PSEUDOCODE
Store student_teacher(Student, Teacher, Subject) facts
show(Student, Teacher, Subject) if matching fact exists
Query a student

5. PROGRAM
Run this program in SWI-Prolog.

```
student_teacher(arun, ravi, cs101).
student_teacher(bala, kumar, cs102).
student_teacher(kumar, suresh, cs103).

show(Student, Teacher, Subject) :-
    student_teacher(Student, Teacher, Subject).
```

6. OUTPUT
```
?- show(arun, Teacher, Subject).
Teacher = ravi,
Subject = cs101.
```

7. RESULT
Thus, the Student-Teacher-Subject database was created in Prolog.

Experiment 20: Planets Database (Prolog)

1. AIM
To create a database containing the names of the eight planets.

2. ALGORITHM
77. Store each planet as a fact.
78. Define a rule to retrieve a planet.
79. Query the rule using a variable.
80. Display all matching planets.

3. CORE LOGIC
Prolog returns each planet fact when the query is asked, one result at a time.

4. PSEUDOCODE
Store planet names as facts
show(X) if planet(X)
Query show(X) and request next results

5. PROGRAM
Run this program in SWI-Prolog; press ; to see the next answer.

```
planet(mercury).
planet(venus).
planet(earth).
planet(mars).
planet(jupiter).
planet(saturn).
planet(uranus).
planet(neptune).

show(X) :-
   planet(X).
```

6. OUTPUT
```
?- show(X).
X = mercury ;
X = venus ;
X = earth ;
X = mars ;
X = jupiter ;
X = saturn ;
X = uranus ;
X = neptune.
```

7. RESULT
Thus, a database of the eight planets was created in Prolog.